

SPECTRUM: LOOPED SELF-DISTILLATION THROUGH PROXIMAL SPECTRAL MODULATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Learning from self-generated solutions can increase a language model’s correctness while narrowing the range of correct solutions it produces. This distinction matters for coding, mathematical reasoning, and other tasks with multiple valid solution paths: the next learning round inherits a distribution of solutions, not an accuracy score. We study this effect through *Looped Self-Distillation*, a modular framework that repeatedly constructs a generation policy, samples a synthetic corpus, and updates a single student. We introduce SPECTRUM, which re-estimates loss-sensitive key/value geometry at each round and converts it into a full-rank proximal spectral modulation. Its continuous, bounded gains shape generation without eliminating any local direction at finite strength; ordinary self-distillation transfers the resulting data distribution into a student that uses standard inference. Over five rounds on MBPP, SPECTRUM retains 89.9% of the initial model’s 64-sample correct-implementation richness, compared with 66.4% for plain self-distillation. Its diversity advantage persists when the number of correct samples is held fixed, and its pass@64 is 2.2 percentage points higher than plain self-distillation. These results establish correct-solution retention as a distinct outcome of repeated self-distillation and identify generation-time spectral modulation as a practical way to improve it.

1 INTRODUCTION

The value of a correct answer is not exhausted by whether it is correct. A programming task can admit implementations with different decompositions, control flow, and data structures; a mathematical problem can admit different derivations. Sampling several valid paths supports search, verification, and the construction of richer training data. This last role becomes consequential when a language model learns from its own outputs: the solutions it produces today become part of the distribution it learns tomorrow.

Self-distillation offers a simple route to such improvement. Recent code-generation methods train a student on outputs sampled from the same model family, making the generation policy an important determinant of learning (Zhang et al., 2026). Repeated generation and training also have established precedents in self-improvement (Zelikman et al., 2022; Gulcehre et al., 2023; Singh et al., 2024). These procedures make a central question unavoidable: *when correctness improves, what happens to the variety within the correct solutions?* Correctness and diversity are recognized as distinct in creative reasoning and code analysis (Hu et al., 2026; Rajput et al., 2025). We focus on their joint evolution across successive student updates.

Our starting observation is a separation between the two outcomes. In a five-round MBPP experiment, plain self-distillation raises pass@1 from 37.94% to 41.45%, while expected correct AST richness at 64 samples falls from 12.802 to 8.506 implementations per task. Figure 1 places this longitudinal phenomenon beside a simple probability example: redistributing probability among correct implementations changes diversity while leaving every pass@ k unchanged. Accuracy therefore cannot determine what variety of correct training material remains available for the next round.

We formalize *Looped Self-Distillation* as an outer learning process with explicit generation and student-update modules. This formulation separates the initial checkpoint, the temporarily modified generation policy, and the native student evaluated after learning. It accommodates different synthetic-data generators without requiring a different adapter for each solution strategy. Within this framework,

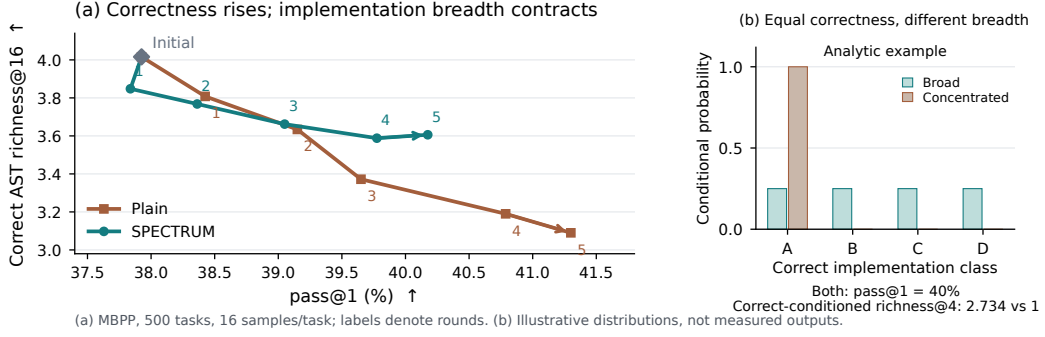


Figure 1: **Correctness does not determine correct-solution diversity.** The measured panels use the completed five-round MBPP experiment, with 16 samples per task at each round. The probability illustration explicitly holds correctness fixed while redistributing mass among correct implementations; it is an analytical example, not an estimated program histogram. Richness counts distinct normalized AST classes; the trajectory displays point estimates at a common 16-sample budget.

SPECTRUM uses the current student’s reference-loss gradients to estimate key/value geometry, then derives a continuous spectral gain from a proximal objective. A finite modulation strength controls how far generation departs from the current student; all local directions retain positive gain. The resulting raw outputs train one LoRA student (Hu et al., 2022), and the geometry is re-estimated at the next round.

The evidence supports a concrete retention benefit. After five rounds, SPECTRUM retains 89.9% of the initial correct AST richness at 64 samples, versus 66.4% for plain self-distillation. It also produces more distinct implementations when compared at the same number of correct samples, and solves more tasks at the largest measured sampling budget. These are outcomes of the final native student, whose inference requires no spectral modification. We organize the study around three contributions:

1. We characterize **correctness–diversity dissociation** in repeated self-distillation, combining a probability decomposition with longitudinal measurements of correct-implementation richness.
2. We formulate **Looped Self-Distillation** as a modular generation–learning framework, making the evolution of both correctness and correct-solution diversity explicit.
3. We introduce **SPECTRUM**, a per-round proximal spectral modulation with a closed form and local retention guarantees, and demonstrate improved implementation-richness retention in the learned student.

2 CORRECTNESS AND CORRECT-SOLUTION RETENTION

For a task x , let $v_x(y) \in \{0, 1\}$ indicate whether output y passes the recorded verifier, and let $\phi(y)$ identify its implementation class. For code we use a conservatively normalized AST fingerprint. Define correctness probability $a_\theta(x) = \Pr_{\pi_\theta}[v_x(y) = 1]$ and conditional class probabilities

$$q_{\theta,j}(x) = \Pr[\phi(y) = j \mid v_x(y) = 1], \quad \Pr[v_x(y) = 1, \phi(y) = j] = a_\theta(x)q_{\theta,j}(x). \quad (1)$$

This factorization separates how often the model succeeds from which implementations it uses when it succeeds. We suppress x below and assume independent samples from a fixed decoding policy.

Proposition 1 (Correctness does not identify conditional richness). *For a fixed correctness probability $a > 0$, $\text{pass}@k = 1 - (1 - a)^k$ is invariant to the conditional class distribution q . The expected number of distinct correct classes in k total draws and in b correct draws is, respectively,*

$$C_k(a, q) = \sum_j [1 - (1 - aq_j)^k], \quad D_b(q) = \sum_j [1 - (1 - q_j)^b]. \quad (2)$$

For $b \geq 2$, distributions with identical $\text{pass}@k$ for every k can have different D_b .

For example, set $a = 0.4$. A model concentrated on one correct implementation has $D_4 = 1$; one uniform over four correct implementations has $D_4 = 4[1 - (3/4)^4] = 2.734375$. Both have $\text{pass}@4$ equal to 0.8704. These occupancy identities motivate measuring the conditional distribution alongside correctness.

We report *correct AST richness@k* and *correct-conditioned AST richness@b*. These descriptive names refer to expected class counts using standard rarefaction estimators (Hurlbert, 1971); C_k and D_b are mathematical shorthand, not additional metrics. The first fixes the total generation budget and the second fixes the number of correct samples. Their exact finite-sample estimators, task eligibility, and relationship to $\text{pass}@k$ appear in Appendix A.

Definition 1 (Correct-solution retention profile). *For students $\theta_0, \dots, \theta_T$ evaluated under one common protocol, the retention profile is the sequence $(a_{\theta_t}, C_k(\theta_t), D_b(\theta_t))_{t=0}^T$. Relative richness retention is $R_{t,k} = C_k(\theta_t)/C_k(\theta_0)$ when the denominator is positive. Correctness–diversity dissociation occurs when correctness increases while the corresponding richness decreases.*

This definition does not prescribe a universal acceptable loss or an optimal modulation strength. Those are empirical operating choices. Retention is a ratio of expected numbers of classes, not the fraction of initial class identities that survives. In particular, D_b controls correct-sample count within each eligible task; population comparisons must also identify the eligible task set.

3 LOOPED SELF-DISTILLATION

Let $\mathcal{Q}$ be a fixed collection of synthesis prompts, $\mathcal{D}_{\text{ref}}$ a fixed calibration set, and θ_t the current native student. A round comprises a generation-policy constructor $\mathcal{A}$, synthetic sampling, and a learner $\mathcal{L}$:

$$\tilde{\pi}_t = \mathcal{A}(\theta_t, \mathcal{D}_{\text{ref}}), \quad \mathcal{D}_t = \{(x, y_{x,s}) : x \in \mathcal{Q}, y_{x,s} \sim \tilde{\pi}_t\}_{s=1}^S, \quad \theta_{t+1} = \mathcal{L}(\theta_t, \mathcal{D}_t). \quad (3)$$

Evaluation always targets $\pi_{\theta_{t+1}}$, with the same decoding settings across methods. The distinction between $\tilde{\pi}_t$ and $\pi_{\theta_{t+1}}$ prevents a temporary decoding benefit from being mistaken for a learned one.

Equation (3) is an operational framework for repeated self-distillation, building on established iterative self-training. Plain self-distillation uses an unmodified generator; a decoding-based method changes the sampling policy; SPECTRUM changes selected generation weights temporarily. A corpus-selection rule can be included in $\mathcal{L}$ when a compared method requires it. Our completed study trains on all raw outputs and warm-starts the current student at every round. The fixed reference set supplies no new human annotations between rounds. Reward-based methods such as UA-RL are useful comparators for the same endpoints, but their policy-gradient updates remain distinct from self-distillation.

4 SPECTRUM: PROXIMAL SPECTRAL MODULATION

4.1 ESTIMATING THE CURRENT STUDENT’S LOSS-SENSITIVE GEOMETRY

For each reference (x_i, y_i^*) , define the mean completion-token negative log likelihood $\ell_i(\theta_t)$. Prompt and padding targets are masked in this calibration loss. For a selected key or value projection ℓ , let $g_{iu}^{(\ell)}$ be the gradient of ℓ_i with respect to its native linear output at nonpadding position u . Following the use of reference-loss gradients to characterize generation-relevant activation directions (Hao et al., 2026), form the uncentered second moment

$$C_{t,\ell} = \frac{1}{M_\ell} \sum_{i,u \text{ nonpad}} g_{iu}^{(\ell)} g_{iu}^{(\ell)\top}, \quad \bar{C}_{t,\ell} = C_{t,\ell} / \lambda_{\max}(C_{t,\ell}). \quad (4)$$

Here M_ℓ counts every included position, including positions with zero gradient. The loss mask and the positions at which gradients are collected are separate choices. Calibration is computed per example before accumulation, making its normalization independent of batching.

The eigenvalues of $\bar{C}$ lie in $[0, 1]$. A large eigenvalue identifies high mean-squared sensitivity of the reference loss along that direction; it does not identify a signed parameter update or a particular algorithm. A finite reference set cannot exhaust all valid implementations. We therefore use its sensitivity information to modulate generation while retaining a positive response in every local direction.

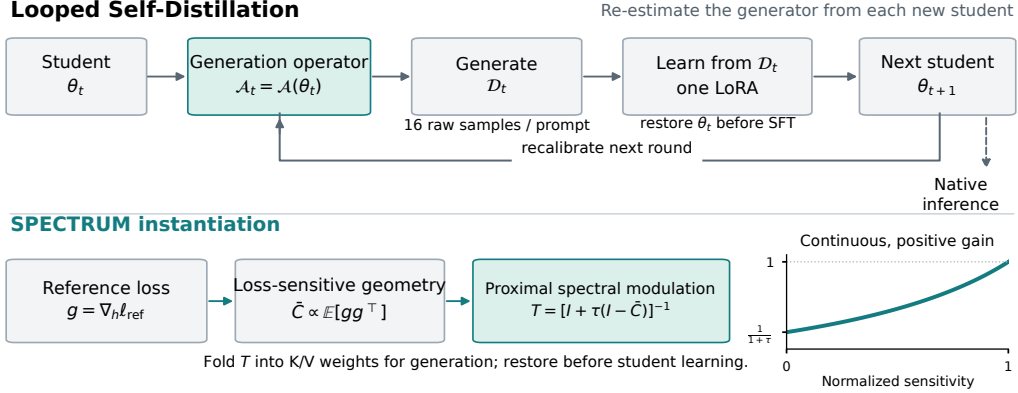


Figure 2: **A modular loop with a SPECTRUM generation module.** Each round begins with a native student. Reference-completion loss defines a gradient second moment, whose normalized spectrum determines the proximal gain. The gain is temporarily folded into K/V weights to generate raw training outputs. Restoring the native weights before single-LoRA learning produces the next student, evaluated through standard inference and used to re-estimate geometry in the following round. The gain inset is analytical.

4.2 A PROXIMAL GENERATION OPERATOR

For a row activation $h \in \mathbb{R}^{1 \times d}$ and modulation strength $\tau \geq 0$, define

$$z^* = \arg \min_z \left\{ \frac{1}{2} \|z - h\|_2^2 + \frac{\tau}{2} z(\mathbf{I} - \bar{C})z^\top \right\} = hT_\tau, \quad T_\tau = [\mathbf{I} + \tau(\mathbf{I} - \bar{C})]^{-1}. \quad (5)$$

The first term anchors generation to the current model; the second attenuates directions less emphasized by the reference geometry. This is a quadratic proximal map (Parikh & Boyd, 2014). If $\bar{C} = U \text{diag}(\mu_j)U^\top$, then

$$T_\tau = U \text{diag} \left(\frac{1}{1 + \tau(1 - \mu_j)} \right) U^\top. \quad (6)$$

The most sensitive directions have gain one. Other gains decrease continuously with τ , whose value is a design parameter studied experimentally. No solution labels, strategy count, or assignment of implementations to adapters is required.

Proposition 2 (Local retention and stability). *Suppose $\bar{C} \succeq 0$ has spectrum in $[0, 1]$ and $0 \leq \tau < \infty$. Equation (5) has a unique solution. Its gains lie in $[1/(1 + \tau), 1]$, and for every local difference δ ,*

$$\frac{\|\delta\|_2}{1 + \tau} \leq \|\delta T_\tau\|_2 \leq \|\delta\|_2, \quad \|T_\tau - \mathbf{I}\|_2 \leq \frac{\tau}{1 + \tau}. \quad (7)$$

For two normalized second moments A, B with spectra in $[0, 1]$,

$$\|T_\tau(A) - T_\tau(B)\|_2 \leq \tau \|A - B\|_2. \quad (8)$$

These properties provide a local design rationale: finite modulation retains distinguishability of local activation states, and the operator depends continuously on normalized calibration statistics without an eigengap assumption. They do not identify activation directions with semantic strategies. The effect after nonlinear generation and learning is evaluated through program outputs. Appendix B provides the proofs and assumptions.

4.3 GENERATING DATA AND LEARNING A NATIVE STUDENT

For $h = xW^\top + b^\top$, replace the selected linear map temporarily by $W' = T_\tau^\top W$ and $b' = T_\tau^\top b$. Then $xW'^\top + b'^\top = hT_\tau$ exactly in real arithmetic. The intervention is applied at native K/V outputs before positional transformations; generation uses ordinary forward execution. After sampling, we restore θ_t and optimize a single LoRA adapter on every raw completion, including incorrect or

Table 1: **Final native-student evaluation after five rounds.** MBPP, 500 tasks, 64 samples per task, training seed 43. Richness is the expected number of distinct correct AST classes. Correct-conditioned richness uses each method’s eligible tasks; paired comparisons use their intersection. Bold identifies the best completed trained-model result. $\times$ denotes a result to be filled after the specified run.

Metric $\uparrow$	Initial	Plain	SSD	UA-RL	SPECTRUM
pass@1 (%)	37.94	41.45	$\times$	$\times$	40.33
pass@8 (%)	61.10	61.86	$\times$	$\times$	61.89
pass@16 (%)	65.69	65.64	$\times$	$\times$	66.23
pass@64 (%)	72.00	70.40	$\times$	$\times$	72.60
Correct AST richness @64	12.802	8.506	$\times$	$\times$	11.510
Relative richness retention (%)	100.0	66.4	$\times$	$\times$	89.9
Correct-conditioned richness @4	3.358	2.856	$\times$	$\times$	3.143
Eligible tasks for previous row	322	321	$\times$	$\times$	323
Solved tasks / 500	360	352	$\times$	$\times$	363

duplicate outputs. The student uses all nonpadding causal targets, distinct from the completion-only calibration mask. The adapter is merged to produce θ_{t+1} .

Re-estimation closes the loop: the next round measures the updated student’s geometry rather than applying a permanently fixed transform. This separates a generation policy that guides data collection from the parameters that ultimately retain the benefit. Table 5 in Appendix C specifies the complete procedure.

5 EXPERIMENTS

5.1 PROTOCOL AND COMPARISONS

The completed experiment uses Qwen2.5-Coder-1.5B-Instruct (Hui et al., 2024) on MBPP (Austin et al., 2021). We preserve its 500 official test tasks and remove training-prompt overlaps before allocating 291 synthesis prompts, 50 calibration references, and 30 validation tasks. Five rounds each generate 16 raw outputs per synthesis prompt and train one LoRA epoch. Methods share prompts, candidate counts, optimization schedules, and native-student evaluation policies. Generated token counts can differ and are reported separately.

Each recorded round has 16 evaluation samples per task; an additional final-checkpoint evaluation uses 64 samples per task. We keep these tiers separate. Correctness is determined by a recorded local Python task-test harness, and AST fingerprints measure implementation structure. The updated experiment suite additionally specifies 64-sample evaluation at every round, three independent training seeds, and cross-benchmark and model-scale extensions. Unavailable results are marked $\times$ in the corresponding tables; they do not contribute to the conclusions below.

The main comparison includes the initial model, plain self-distillation, SSD’s decoding recipe, UA-RL, and SPECTRUM. SSD and UA-RL are listed with $\times$ until their common-protocol five-round runs are available. UA-RL is an adapted policy-gradient comparator with semantic strategy clustering, not an SFT variant. Simple operator replacements are evaluated separately as design ablations. Appendix D gives the hyperparameters and Appendix F defines the extension matrix.

5.2 HOW MUCH CORRECT-IMPLEMENTATION RICHNESS IS RETAINED?

Table 1 shows that SPECTRUM retains substantially more correct implementation variety than plain self-distillation. Its 64-sample richness is 11.510, compared with 8.506, a relative increase of 35.3%. Normalizing by the initial model gives retention rates of 89.9% and 66.4%. The distinction matters because plain self-distillation improves single-sample accuracy more strongly while retaining fewer correct implementations.

The higher richness is accompanied by better large-budget success: SPECTRUM solves 363 of 500 tasks versus 352 for plain self-distillation. The paired pass@64 difference is +2.2 percentage points,

Table 2: **Paired SPECTRUM minus Plain comparisons.** Final 64-sample evaluation. Intervals resample tasks, not training seeds. D_b comparisons use common eligible tasks; C_{64} and $\text{pass}@k$ use all 500 tasks. Mathematical symbols are defined in Section 2.

Endpoint	Difference	95% interval	Paired tasks
Correct AST richness @64 (C_{64})	+3.004	[2.568, 3.494]	500
Correct-conditioned richness @4 (D_4)	+0.295	[0.249, 0.343]	316
Correct-conditioned richness @8 (D_8)	+0.780	[0.665, 0.889]	296
Correct-conditioned richness @16 (D_{16})	+1.816	[1.582, 2.055]	254
$\text{pass}@1$ (percentage points)	-1.116	[-1.61, -0.63]	500
$\text{pass}@64$ (percentage points)	+2.200	[0.40, 4.20]	500

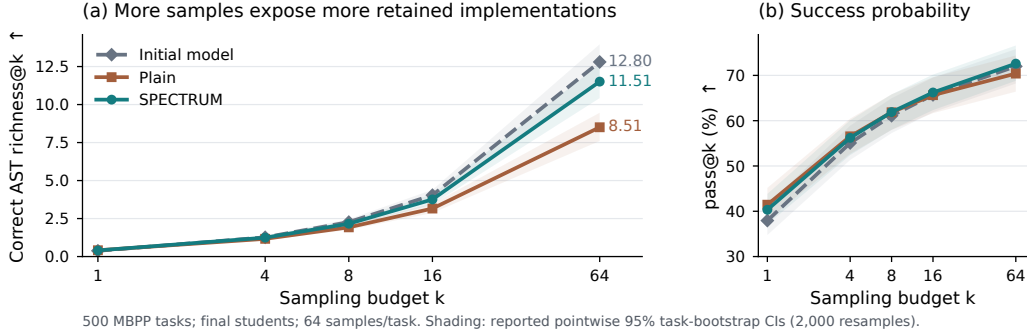


Figure 3: **Sampling budget exposes different aspects of the learned distribution.** Expected correct AST richness and $\text{pass}@k$ are estimated from the same final 64-sample pool, at the recorded budgets $k \in \{1, 4, 8, 16, 64\}$. All-task means use 500 MBPP tasks. Richness and correctness have separate axes and units.

with a 95% task-bootstrap interval of [0.4, 4.2]. Its $\text{pass}@1$ is 1.12 points lower than plain self-distillation and 2.39 points above the initial model. Thus the measured benefit is retention and large-budget search performance, with an explicit single-sample accuracy trade-off.

5.3 DOES THE GAIN PERSIST AT A FIXED CORRECT-SAMPLE COUNT?

More distinct correct outputs could arise simply from sampling more correct programs. Correct-conditioned richness removes this count difference within a task. Table 2 reports paired comparisons on tasks for which both models have enough correct samples. SPECTRUM improves richness at budgets of four, eight, and sixteen correct draws. The four-draw gain is 0.295 AST classes per task; at sixteen correct draws the gain is 1.816. The positive effects therefore concern the implementation distribution within correct outputs, in addition to unconditional success.

5.4 WHAT CHANGES WITH SAMPLING BUDGET AND LEARNING ROUND?

Figure 3 evaluates different budgets using the same final pool of 64 samples per task. At small budgets, correctness has a strong influence on observed richness. At larger budgets, repeated visits to common implementations leave more room for the conditional distribution to matter. SPECTRUM has greater richness than plain self-distillation at each displayed multi-sample budget and higher $\text{pass}@64$. The curves use only recorded budgets; the connecting segments do not imply an unmeasured scaling law.

Figure 4 examines the recorded 16-sample evaluations across five rounds. Correct-conditioned richness falls from an initial 3.310 to 2.747 under plain self-distillation and 2.997 under SPECTRUM. The latter retains more correct-conditioned richness at every recorded round. Paired differences from the initial model further distinguish the methods while holding the task set fixed within each contrast. Eligibility may vary between rounds, so these are pointwise longitudinal summaries rather than a single fixed-cohort estimate.

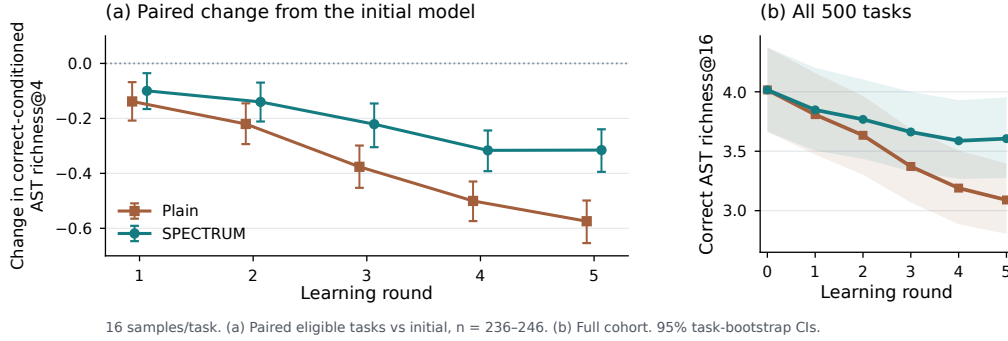


Figure 4: **The retention benefit persists through the loop.** All observations use the per-round 16-sample protocol. Paired changes relative to the initial model use the common eligible tasks for each contrast; absolute richness uses its stated task population. Intervals are pointwise 95% task-bootstrap intervals. The 64-sample final results are reported separately in Table 1.

Table 3: **Generation-design ablations after five rounds.** Same MBPP protocol; 64 final samples per task. Identity is plain self-distillation. Rank truncation is a local operator-design control. Other rows await execution. The two matched controls match operator-level quantities, not output KL or generated token budgets.

Generator design	pass@1 (%)	pass@64 (%)	AST richness @64	Retention (%)
Identity	41.45	70.40	8.506	66.4
Rank truncation (half directions)	41.77	70.20	8.388	65.5
SPECTRUM ($\tau = 1$)	40.33	72.60	11.510	89.9
Randomized eigenvectors	x	x	x	x
Matched isotropic attenuation	x	x	x	x
Fixed first-round geometry	x	x	x	x

5.5 DESIGN ABLATIONS AND FURTHER COMPARISONS

We separate method comparisons from simple generation-operator controls. Table 3 includes a rank-truncation design that keeps half the calibrated directions and sets the others to zero. It uses the same calibration and learning protocol as SPECTRUM and is treated as an ablation, not as a complete reproduction of a prior method. Its final richness retention is 65.5%, compared with 89.9% for SPECTRUM. The continuous operator raises richness by 3.122 classes and pass@64 by 2.4 points, while its pass@1 is 1.43 points lower.

Three additional controls target distinct explanations: randomized eigenvectors preserve the gain spectrum but change its orientation; isotropic attenuation matches the operator’s Frobenius distance from identity without directional selectivity; fixed geometry reuses the first-round statistic. A modulation-strength sweep studies the operating point. These controls are specified, with unavailable results marked x, rather than inferred from the completed soft-versus-truncation comparison.

The extension protocol evaluates four additional code benchmarks and models at 3B, 7B, and an additional decoder family. It distinguishes transfer of an MBPP-trained student from training anew on another benchmark. Resource and sample-length tables accompany the comparisons, since equal candidate counts do not ensure equal generated tokens or wall time. Appendix F contains the full matrices, and Appendix G reports available resource counts.

5.6 RESOURCES AND SAMPLE LENGTH

The benefit comes with a measurable change in the generated corpus. Across the five completed rounds, SPECTRUM produces 5.34 million training-generation tokens, compared with 4.86 million for plain self-distillation, a 9.9% increase at the same 23,280 candidates. Mean output length rises from 208.76 to 229.48 tokens (Table 4). Reporting both quantities distinguishes a matched sampling

Table 4: Available training-generation resources over five rounds. Mean length is generated tokens divided by all 23,280 records. Time, peak memory, and judge usage require the corresponding archived runtime records or new runs; $\times$ is not zero.

Method	Records	Generated tokens	Mean tokens	GPU hours	Peak GB
Plain	23,280	4,859,928	208.76	$\times$	$\times$
SSD	$\times$	$\times$	$\times$	$\times$	$\times$
UA-RL (adapted)	$\times$	$\times$	$\times$	$\times$	$\times$
SPECTRUM	23,280	5,342,368	229.48	$\times$	$\times$
Rank-truncation control	23,280	4,727,089	203.05	$\times$	$\times$

allocation from equal realized token usage. It also motivates the fixed-correct-count analysis: that comparison measures distinct correct structures at the same number of successful draws rather than treating additional outputs as diversity.

Calibration adds reference-gradient collection and module-wise eigendecompositions. For an output dimension d and M accumulated positions, the second-moment and factorization costs scale as $O(Md^2)$ and $O(d^3)$, respectively. Temporary weight folding does not introduce a separate operator into the generation forward pass, and the final student has the original architecture. Actual time and memory depend on the model and hardware and remain separately recorded quantities; the current aggregate bundle supplies token counts, while the expanded runs export stage-wise runtime and memory.

6 RELATED WORK

Self-generated data and iterative improvement. STaR, ReST, and ReST-EM repeatedly generate training material and update a model (Zelikman et al., 2022; Gulcehre et al., 2023; Singh et al., 2024); SD-Zero studies iterative self-distillation with teacher synchronization (He et al., 2026). ReST-EM also examines later-round regressions. SSD studies self-distillation from raw code outputs and analyzes precision and exploration (Zhang et al., 2026); self-policy distillation uses capability-selective activation geometry to guide generation (Hao et al., 2026). Our framework makes generation and learning explicit and evaluates how within-correct implementation richness evolves in the resulting native students.

Diversity, recursive collapse, and recurrent computation. UA-RL rewards underrepresented high-level strategies through group-relative policy optimization (Hu et al., 2026). Its reference-strategy coverage differs from our expected count of observed correct implementation classes. Functionally correct programs can also differ behaviorally and algorithmically (Rajput et al., 2025). Recursive-data-collapse research studies distributional degradation under repeated model-generated training, and accumulation can mitigate that degradation (Shumailov et al., 2024; Gerstgrasser et al., 2024). We examine a conditional, executable-task outcome that can deteriorate while task correctness rises, using generation modulation rather than historical data accumulation. Finally, looped or recurrent-depth transformers repeat computation within an inference process (Dehghani et al., 2019; Geiping et al., 2025); our loop updates model parameters between synthetic-data rounds.

Spectral control. Activation editing and conceptors provide spectral or soft-subspace mechanisms for controlling models (Qiu et al., 2024; Postmus & Abreu, 2024). SPECTRUM combines a reference-loss second moment with a specific proximal gain and repeatedly distills its generation distribution into a native student. The per-round recalibration and student-level evaluation connect spectral control to correct-solution retention throughout learning.

7 DISCUSSION AND CONCLUSION

Repeated self-distillation should be evaluated as a change in a solution distribution, not only in a success probability. Our completed experiment shows that single-sample accuracy can rise as correct implementation richness contracts, and that per-round proximal spectral modulation substantially

reduces that contraction in the final native student. Fixed-correct-count comparisons connect the gain to variety within successful outputs.

The current measured scope is one model, one benchmark, and one training seed; the extension tables separate planned comparisons from completed evidence. AST classes represent implementation structure, not audited algorithm identities, and correctness is relative to the available tests. The local operator guarantees motivate SPECTRUM but do not imply a semantic-diversity theorem for the full network. Within this scope, the results support a concrete design principle: treat the diversity of correct synthetic solutions as an outcome to retain throughout self-improvement.

REPRODUCIBILITY STATEMENT

Appendices A–G give estimator definitions, proofs, calibration and training details, evaluator protocols, full longitudinal results, and resource accounting. The accompanying code exports per-task estimates, task-bootstrap intervals, provenance, and resumable round checkpoints. Numerical cells marked $\times$ identify the explicitly specified extension runs whose results are not yet available.

AI USE STATEMENT

AI assistants were used to assist with literature discovery, mathematical exposition, manuscript drafting, figure code, and experiment software. No experimental scores were generated by an AI assistant to fill missing results; pending entries are marked explicitly.

REFERENCES

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>. arXiv preprint.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>. arXiv preprint.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1807.03819>.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach, 2025. URL <https://arxiv.org/abs/2502.05171>.
- Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Henry Sleight, John Hughes, Tomasz Korbak, Rajashree Agrawal, Dhruv Pai, Andrey Gromov, Daniel A. Roberts, Diyi Yang, David L. Donoho, and Sanmi Koyejo. Is model collapse inevitable? breaking the curse of recursion by accumulating real and synthetic data, 2024. URL <https://arxiv.org/abs/2404.01413>. arXiv preprint.
- Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling, 2023. URL <https://arxiv.org/abs/2308.08998>.
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. DeepSeek-Coder: When the large language model meets programming—the rise of code intelligence, 2024. URL <https://arxiv.org/abs/2401.14196>.
- Guangya Hao, Yitong Shang, Yunbo Long, Zhuokai Zhao, and Hanxue Liang. Self-policy distillation via capability-selective subspace projection, 2026. URL <https://arxiv.org/abs/2605.22675>. arXiv preprint.
- Yinghui He, Simran Kaur, Adithya Bhaskar, Yongjin Yang, Jiarui Liu, Narutatsu Ri, Liam Fowl, Abhishek Panigrahi, Danqi Chen, and Sanjeev Arora. Self-distillation zero: Self-revision turns binary rewards into dense supervision, 2026. URL <https://arxiv.org/abs/2604.12002>.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS, 2021. URL <https://arxiv.org/abs/2105.09938>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.

- Zhiyuan Hu, Yucheng Wang, Yufei He, Jiaying Wu, Yilun Zhao, See-Kiong Ng, Cynthia Breazeal, Anh Tuan Luu, Hae Won Park, and Bryan Hooi. Rewarding the rare: Uniqueness-aware RL for creative problem solving in LLMs. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 39765–39790, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.1982. URL <https://aclanthology.org/2026.findings-acl.1982/>.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-Coder technical report, 2024. URL <https://arxiv.org/abs/2409.12186>. arXiv preprint.
- Stuart H. Hurlbert. The nonconcept of species diversity: A critique and alternative parameters. *Ecology*, 52(4):577–586, 1971. doi: 10.2307/1934145. URL <https://esajournals.onlinelibrary.wiley.com/doi/10.2307/1934145>.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode, 2022. URL <https://arxiv.org/abs/2203.07814>.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html.
- Neal Parikh and Stephen Boyd. Proximal algorithms. *Foundations and Trends in Optimization*, 1(3), 2014. doi: 10.1561/24000000003. URL https://web.stanford.edu/~boyd/papers/prox_algs.html.
- Joris Postmus and Steven Abreu. Steering large language models using conceptors: Improving addition-based activation engineering, 2024. URL <https://arxiv.org/abs/2410.16314>. arXiv preprint; presented at the MINT workshop at NeurIPS 2024.
- Yifu Qiu, Zheng Zhao, Yftah Ziser, Anna Korhonen, Edoardo M. Ponti, and Shay B. Cohen. Spectral editing of activations for large language model alignment. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-1815. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/684c59d614fe6ae74a3be8c3ef07e061-Abstract-Conference.html.
- Prateek Rajput, Abdoul Aziz Bonkougou, Yewei Song, Abdoul Kader Kabore, Iyiola E. Olatunji, Jacques Klein, and Tegewende Bissyande. Dynamic stability of LLM-generated code, 2025. URL <https://arxiv.org/abs/2511.07463>.
- Ilya Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024. doi: 10.1038/s41586-024-07566-y. URL <https://www.nature.com/articles/s41586-024-07566-y>.
- Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, Abhishek Kumar, Alex Alemi, Alex Rizkowsky, Azade Nova, Ben Adlam, Bernd Bohnet, Gamaleldin Elsayed, Hanie Sedghi, Igor Mordatch, Isabelle Simpson, Izzeddin Gur, Jasper Snoek, Jeffrey Pennington, Jiri Hron, Kathleen

Kenealy, Kevin Swersky, Kshiteej Mahajan, Laura Culp, Lechao Xiao, Maxwell L. Bileschi, Noah Constant, Roman Novak, Rosanne Liu, Tris Warkentin, Yundi Qian, Yamini Bansal, Ethan Dyer, Behnam Neyshabur, Jascha Sohl-Dickstein, and Noah Fiedel. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2312.06585>.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning, 2022. URL <https://arxiv.org/abs/2203.14465>.

Ruixiang Zhang, Richard He Bai, Huangjie Zheng, Navdeep Jaitly, Ronan Collobert, and Yizhe Zhang. Embarrassingly simple self-distillation improves code generation, 2026. URL <https://arxiv.org/abs/2604.01193>. arXiv preprint.

A METRIC DEFINITIONS AND PROBABILITY ANALYSIS

A.1 THE OBJECT MEASURED

A task is a prompt together with its recorded verification procedure. Let $V = 1$ denote passing that procedure and $J = \phi(Y)$ the implementation fingerprint. Equation (1) is the conditional-probability factorization $\Pr(V = 1, J = j) = \Pr(V = 1) \Pr(J = j \mid V = 1)$. Correct-conditioned quantities require $a > 0$. If a model never succeeds on a task, unconditional richness is zero and the conditional distribution is undefined, rather than being assigned an arbitrary zero-diversity distribution.

For countably many correct classes, all sums below converge: $1 - (1 - p)^k \leq kp$ bounds $C_k \leq ka$ and $D_b \leq b$. With J positive-probability classes, $C_k \leq \min(J, ka)$ and $D_b \leq \min(J, b)$. A finite sample estimates the number of classes encountered at the specified budget, not the total support size of the model.

A.2 PROOF OF PROPOSITION 1

Let I_j indicate whether at least one of k draws is a correct output from class j . Then $\mathbb{E}I_j = 1 - (1 - aq_j)^k$, and linearity of expectation gives C_k . For b independent draws from the conditional correct-output distribution, the same argument gives D_b . The event that all k unconditional draws fail has probability $(1 - a)^k$, giving $\text{pass}@k$. Holding a fixed preserves this probability for every k , whereas taking $q = (1, 0, \dots)$ or a uniform distribution on $J \geq 2$ classes gives respectively $D_b = 1$ and $D_b = J[1 - (1 - 1/J)^b] > 1$ for $b \geq 2$. Thus correctness fails to identify conditional richness. $\square$

The uniform example approaches J as b increases without bound. It is not equal to $\min(J, b)$ at finite budget, because repeated draws remain possible. For $b = 1$, every defined D_1 equals one, so it cannot distinguish conditional diversity.

A.3 FINITE-SAMPLE ESTIMATORS AND RAREFACTION

For one task, suppose n outputs contain m verified-correct outputs, with m_j in correct fingerprint class j and $\sum_j m_j = m$. The estimators are

$$\widehat{\text{pass}@k} = 1 - \frac{\binom{n-m}{k}}{\binom{n}{k}}, \quad 1 \leq k \leq n, \quad (9)$$

$$\widehat{C}_k = \sum_j \left[1 - \frac{\binom{n-m_j}{k}}{\binom{n}{k}} \right], \quad 1 \leq k \leq n, \quad (10)$$

$$\widehat{D}_b = \sum_j \left[1 - \frac{\binom{m-m_j}{b}}{\binom{m}{b}} \right], \quad 1 \leq b \leq m. \quad (11)$$

We set $\binom{a}{b} = 0$ for nonnegative integers $a < b$. The first is the standard $\text{pass}@k$ estimator (Chen et al., 2021). The other two are individual-based rarefaction estimators applied to correct implementation classes (Hurlbert, 1971). Selecting a uniform subset of k observed outputs misses class j with probability $\binom{n-m_j}{k} / \binom{n}{k}$; the analogous calculation among only correct outputs yields Equation (11). Conditional on the observed pool these are exact expected subset counts; under independent sampling they estimate the population quantities at the same draw budget.

At $k = n$, $\widehat{C}_n$ is exactly the number of distinct observed correct fingerprints. At $k = 1$, $\widehat{C}_1 = m/n = \widehat{\text{pass}@1}$. At $b = m > 0$, $\widehat{D}_m$ is the same observed class count, but D_m is available only when that many correct samples exist. This is why a 64-correct-draw analysis would condition on a small and method-dependent subset of a 64-total-draw evaluation; we emphasize $b \in \{4, 8, 16\}$ instead.

A.4 AST, EXACT-CODE, AND CONTROL-FLOW VIEWS

The primary fingerprint parses the extracted Python program and hashes a conservatively normalized AST. Formatting and docstrings do not affect it. Simple local identifiers are normalized when scope handling is unambiguous; function names, external identifiers, literals, and uncertain scopes are

retained. It is not an algorithm-equivalence oracle. Exact-code fingerprints and a control-flow-node skeleton supply secondary views. Parse failures are recorded and do not silently become new correct classes.

In the recorded per-round evaluation, the fifth-round SPECTRUM-minus-truncation comparison improves exact-code four-correct-draw richness by 0.312 with interval $[0.253, 0.377]$ and AST richness by 0.293 with interval $[0.233, 0.353]$. The coarser control-flow result is 0.042 with interval $[-0.011, 0.095]$. Thus the evidence is strongest for implementation structure and does not establish a uniform increase in high-level algorithm families.

A.5 TASK AGGREGATION, PAIRED CONTRASTS, AND UNCERTAINTY

Unconditional correctness and C_k are macro-averaged over all evaluation tasks with a complete sample pool. For D_b , define $\mathcal{E}_b(M) = \{x : m_x(M) \geq b\}$ for model M . Its reported marginal mean averages over this set and prints its size. A comparison of models A, B instead uses $\mathcal{E}_b(A) \cap \mathcal{E}_b(B)$ and averages within-task differences. The difference between two marginal means with different eligible sets is not the paired contrast.

Reported intervals use 2,000 task-bootstrap replicates at the 2.5th and 97.5th percentiles. Paired resampling preserves method pairing within each selected task; conditional metrics retain their declared eligibility rule and report valid-replicate counts. These intervals quantify uncertainty across tasks for a fixed trained model. They do not estimate variation across training seeds or create independent replicates from correlated learning rounds. New multi-seed outputs retain task and seed indices separately. A fixed-cohort longitudinal analysis uses the common eligible task intersection across all specified models and rounds; it requires per-task values and cannot be reconstructed from aggregate means alone.

B PROXIMAL OPERATOR PROPERTIES AND DERIVATIONS

B.1 CALIBRATION LOSS AND SECOND-MOMENT INTERPRETATION

Let $\mathcal{T}_i$ be the selected nonpadding completion targets for reference i . Calibration uses

$$\ell_i(\theta) = -\frac{1}{|\mathcal{T}_i|} \sum_{u \in \mathcal{T}_i} \log \pi_\theta(y_{iu}^* \mid x_i, y_{i, < u}^*). \quad (12)$$

This mean is computed separately for every example. Gradient vectors at all nonpadding K/V-output positions, including prompt positions, contribute to Equation (4). The statistic is an uncentered second moment. A large eigenvalue measures average squared directional sensitivity, so gradients of opposite sign both contribute positively. No claim about signed agreement follows from this statistic. An empty or zero-signal calibration is rejected because division by a positive maximum eigenvalue would otherwise be undefined.

B.2 PROOF OF PROPOSITION 2

The Hessian of the objective in Equation (5) is $H = \mathbf{I} + \tau(\mathbf{I} - \overline{C})$. Its eigenvalues are $1 + \tau(1 - \mu_j) \in [1, 1 + \tau]$, so H is positive definite and the minimizer is unique. Setting the derivative to zero gives $zH = h$, hence $z = hH^{-1}$. Diagonalization gives Equation (6). Since T_τ is symmetric positive definite, its singular values equal its gains and yield

$$(1 + \tau)^{-1} \|\delta\|_2 \leq \|\delta T_\tau\|_2 \leq \|\delta\|_2. \quad (13)$$

Furthermore,

$$\|T_\tau - \mathbf{I}\|_2 = \max_j \frac{\tau(1 - \mu_j)}{1 + \tau(1 - \mu_j)} \leq \frac{\tau}{1 + \tau}. \quad (14)$$

For $\tau > 0$, equality in the last bound requires a zero eigenvalue of $\overline{C}$. The general statement is an inequality. The condition number of T_τ is at most $1 + \tau$, and $T_0 = \mathbf{I}$.

For normalized PSD matrices A, B , write $H_A = \mathbf{I} + \tau(\mathbf{I} - A)$ and similarly H_B . The resolvent identity gives

$$T_\tau(A) - T_\tau(B) = H_A^{-1}(H_B - H_A)H_B^{-1} = \tau T_\tau(A)(A - B)T_\tau(B). \quad (15)$$

Both outer factors have norm at most one, proving Equation (8). This proof uses no eigengap. It is a stability statement about normalized matrices; converting error in an unnormalized second moment to error in $\overline{C}$ additionally requires control of its normalization. $\square$

B.3 EXACT FOLDING AND ITS SCOPE

For the row-vector convention $h = xW^\top + b^\top$, the modified activation is

$$hT = xW^\top T + b^\top T = x(T^\top W)^\top + (T^\top b)^\top. \quad (16)$$

Thus folding requires left-multiplication of the output dimension of the stored weight, not right-multiplication of the input dimension. The bias is transformed when present. The code snapshots and restores the original tensors even if generation fails. Calibration uses explicit temporary linear wrappers for gradient access; neither generation nor native-student inference registers activation hooks.

The identity holds at the selected native linear output before RoPE or other downstream transformations. It does not assert that a pre-RoPE intervention equals one applied after RoPE. Positive local singular values likewise do not imply positive probability for every semantic strategy after the entire nonlinear network, decoding procedure, and student update. The local propositions and the output-space measurements serve different evidentiary roles.

B.4 DESIGN CONTROLS

Let P_r project onto the leading r eigenvectors of $\overline{C}$, and let s_j be SPECTRUM’s gains. Rank truncation uses P_r . Randomized-eigenvector modulation uses $Q \text{diag}(s_j) Q^\top$ with a seeded Haar orthogonal Q , preserving the gains while changing their orientation. Write $d_F = \|T_r - \mathbf{I}\|_F$. Matched isotropic attenuation uses $(1 - d_F/\sqrt{d})\mathbf{I}$. An optional matched two-level blend uses $P_r + \rho(\mathbf{I} - P_r)$, where $\rho = 1 - d_F/\sqrt{d} - r$ if this lies in $[0, 1]$; an infeasible match is rejected. These constructions match a spectrum or operator distance, not an activation-weighted distortion, output KL, or computational budget. Fixed geometry reuses first-round calibration; all other SPECTRUM rounds re-estimate it.

C IMPLEMENTATION AND COMPLETE LOOP ALGORITHM

Table 5: Looped Self-Distillation with the SPECTRUM generation module.

Inputs: native student θ_0 ; synthesis prompts $\mathcal{Q}$; fixed reference set $\mathcal{D}_{\text{ref}}$; rounds T ; samples per prompt S ; strength τ ; common evaluation protocol.
For $t = 0, \dots, T - 1$:
1. Save the native student state θ_t .
2. On fixed references, compute completion-masked losses and K/V-output gradients; accumulate $C_{t,\ell}$ over all nonpadding positions.
3. Normalize each nonzero second moment and build $T_{t,\ell} = [\mathbf{I} + \tau(\mathbf{I} - \overline{C}_{t,\ell})]^{-1}$.
4. Temporarily fold every selected operator into its K/V weight and bias.
5. Sample S raw completions per synthesis prompt, retaining every output and its generation metadata.
6. Restore θ_t ; initialize one LoRA adapter; train one epoch on the entire raw corpus using all nonpadding targets.
7. Merge the adapter into θ_{t+1} and save the native checkpoint.
8. Evaluate θ_{t+1} without modulation; save outputs, per-task counts, richness estimates, task-bootstrap intervals, and resources.
Return: native student θ_T and the complete retention profile.

The synthesis task order and reference split are fixed independently of the test outputs. Each round’s model and corpus are separately identified. Resume checks bind a stage to the configuration, model checkpoint, task hashes, and implementation provenance. Checkpoint retention is set to all rounds in the expanded protocol so that later evaluation can increase sampling without retraining the students.

D DATA, OPTIMIZATION, AND EVALUATION PROTOCOLS

Table 6: Completed five-round MBPP experiment.

Component	Recorded setting
Model	Qwen2.5-Coder-1.5B-Instruct; BF16; SDPA attention
Data	291 synthesis / 50 calibration / 30 validation / 500 official test tasks; split seed 42
Rounds and training seed	Five rounds; seed 43
Generation	16 raw training completions per task per round; temperature 0.8; top- p 0.95; top- k 0; maximum 512 new tokens
Calibration	Middle and final decoder layers; K/V native linear outputs; 50 references; length 1536; completion-only loss
Spectral setting	$\tau = 1$; rank-truncation control retains half the directions
LoRA	Q/K/V/O modules; rank 8; alpha 8; dropout 0.05
Optimization	One epoch; learning rate 10^{-5} ; weight decay 0.01; warmup ratio 0.03; gradient clip 1; microbatch 1; accumulation 16
Student loss	All nonpadding causal targets; maximum sequence length 1536; gradient checkpointing
Evaluation samples	16 per task at every recorded round; 64 per task for initial and final checkpoints
Verification	Recorded local Python 3.11 harness; first fenced-code extraction; task setup and tests; timeout 5 seconds; eight workers
Uncertainty	2,000 task-bootstrap replicates; pointwise 95% intervals

Data integrity. Official MBPP test rows are preserved. Original prompt text is normalized for duplicate detection; prompt overlaps are removed from the training pool before assigning synthesis, calibration, and validation sets. Function signatures exposed to generation are derived from the reference interface, not the reference implementation body. Test-task references are not used to calibrate or train the model. All raw synthesis completions enter SFT, including incorrect, empty, and duplicate outputs; correctness filtering is not part of this completed protocol.

Generation and verification. Every method uses the same tokenizer chat template for a task. Program extraction and the verifier are shared within each comparison. The completed MBPP result is tied to its recorded local task tests and is not an EvalPlus result. For the additional HumanEval+ evaluation, the extended official EvalPlus checks are applied to the saved samples through the benchmark bridge. Other adaptations are named explicitly in Appendix F. Generation and verification records remain separate so that correctness decisions and fingerprints can be audited without rerunning model inference.

SSD and UA-RL. The SSD comparator uses temperature 1.5, top- p 0.8, and top- k 20 for synthesis within the same outer-round interface. Its final evaluation uses the common native policy (0.8, 0.95, and 0, respectively); its five-round results are pending. UA-RL uses a larger-model semantic strategy judge to group outputs for each prompt and weights standardized rewards by cluster rarity (Hu et al., 2026). For a group of G sampled outputs, rewards r_i , and cluster frequencies f_i , the adapted baseline uses

$$A_i = f_i^{-\alpha} \frac{r_i - \bar{r}}{\text{std}(r) + \epsilon}, \quad 0 \leq \alpha \leq 1. \quad (17)$$

Clustering includes the complete group; the factor multiplies positive and negative advantages. AST fingerprints are not substituted for semantic clusters. The adapter records judge calls and tokens separately and retains the policy-gradient objective and KL term. It is labeled an adaptation to our task, budget, and evaluation protocol, rather than an official code reproduction. Candidate budgets and optimizer budgets are reported separately across SFT and RL methods.

E COMPLETE LONGITUDINAL AND PAIRED RESULTS

Table 7 contains the recorded 16-sample trajectories. Base and final 64-sample results come from a different sampling tier and should not be joined to these trajectories. Pointwise intervals and eligible

counts are retained in the underlying report and figure-data files. The expanded exporter additionally stores per-task values and common-cohort comparisons; it does not invent historical per-task values omitted from a compact result bundle.

Table 7: Recorded per-round native-student results, 500 MBPP tasks and 16 samples per task. D_4 uses four correct draws and reports its eligible task count. The rank-truncation rows are included as design controls.

Method	Round	pass@1 (%)	AST richness @16	D_4	Eligible
Initial	0	37.92	4.016	3.310	262
Plain	1	38.42	3.808	3.173	261
Plain	2	39.15	3.634	3.080	269
Plain	3	39.65	3.372	2.895	257
Plain	4	40.79	3.190	2.803	270
Plain	5	41.30	3.090	2.747	268
Rank truncation	1	37.92	3.692	3.144	257
Rank truncation	2	39.20	3.592	3.059	267
Rank truncation	3	39.40	3.314	2.889	256
Rank truncation	4	41.09	3.162	2.774	266
Rank truncation	5	41.35	3.066	2.690	262
SPECTRUM	1	37.84	3.848	3.222	261
SPECTRUM	2	38.36	3.768	3.159	259
SPECTRUM	3	39.05	3.662	3.063	262
SPECTRUM	4	39.77	3.588	2.991	263
SPECTRUM	5	40.17	3.606	2.997	263

Table 8: Final 64-sample paired SPECTRUM minus rank-truncation design-control results. These comparisons are separate from the main method comparison.

Endpoint	Difference	95% interval	Paired tasks
AST richness @64	+3.122	[2.686, 3.596]	500
Correct-conditioned richness @4	+0.334	[0.290, 0.379]	318
Correct-conditioned richness @8	+0.875	[0.773, 0.979]	295
Correct-conditioned richness @16	+1.998	[1.792, 2.218]	254
pass@1 (percentage points)	-1.434	[-1.89, -0.98]	500
pass@64 (percentage points)	+2.400	[0.60, 4.40]	500

The main result is distinct from the earlier 64-task seed-42 pilot and a ten-round smoke run. The pilot helped select the current direction; it is not another full benchmark seed. The smoke run verifies execution on a tiny task set and supplies no evidence for a ten-round research claim. Neither is pooled with the completed 500-task experiment.

F EXTENDED EXPERIMENT MATRIX

F.1 INDEPENDENT SEEDS AND MODULATION STRENGTH

The confirmation runs in Table 9 use seeds 42, 43, and 44, with 64 samples per task at every round. The historical seed-43 experiment remains identified by its original 16-sample longitudinal protocol and its 64-sample final supplement; new runs have separate provenance. Across-seed variability concerns independent trained checkpoints, not task-bootstrap replicates. Table 10 separates modulation-strength sensitivity from seed confirmation.

F.2 FIVE BENCHMARKS WITH A COMMON IMPLEMENTATION-LEVEL QUESTION

We keep the primary extension in Python code so that correctness is executable and the implementation fingerprint has the same interpretation. Table 11 lists MBPP (Austin et al., 2021), HumanEval+ (Liu et al., 2023), APPS introductory (Hendrycks et al., 2021), CodeContests (Li et al., 2022), and a pinned LiveCodeBench release (Jain et al., 2024). The latter four first evaluate transfer from the

Table 9: Independent training-seed confirmation matrix. Entries marked $\times$ require the new common-protocol runs. Historical seed 43 is reported separately rather than silently substituted into a different protocol.

Method	Seed	pass@1	pass@64	AST richness @64	D_4
Plain	42 / 43 / 44	$\times$	$\times$	$\times$	$\times$
SSD	42 / 43 / 44	$\times$	$\times$	$\times$	$\times$
UA-RL (adapted)	42 / 43 / 44	$\times$	$\times$	$\times$	$\times$
SPECTRUM	42 / 43 / 44	$\times$	$\times$	$\times$	$\times$

Table 10: Modulation-strength study on MBPP. The known $\tau = 1$ row is the historical final-checkpoint result; other cells remain pending. Uniform-schedule reruns are stored separately.

τ	pass@1 (%)	pass@64 (%)	AST richness @64	D_4
0 (identity, historical Plain)	41.45	70.40	8.506	2.856
0.25	$\times$	$\times$	$\times$	$\times$
0.5	$\times$	$\times$	$\times$	$\times$
1 (historical SPECTRUM)	40.33	72.60	11.510	3.143
2	$\times$	$\times$	$\times$	$\times$
4	$\times$	$\times$	$\times$	$\times$

frozen MBPP-trained students; they are not portrayed as separate training datasets or completed mathematical-reasoning experiments. The converters record source revisions, task IDs, selected subsets, prompt changes, and unsupported records.

Table 11: Cross-benchmark native-student evaluation matrix. Each cell is pass@1 / pass@64 / correct AST richness@64; $\times$ denotes pending measurement. All transferred students are trained on MBPP.

Benchmark	Initial	Plain	SPECTRUM
MBPP	37.94/72.00/12.80	41.45/70.40/8.51	40.33/72.60/11.51
HumanEval+	$\times$	$\times$	$\times$
APPS introductory (adapted)	$\times$	$\times$	$\times$
CodeContests (adapted)	$\times$	$\times$	$\times$
LiveCodeBench (adapted)	$\times$	$\times$	$\times$

For standard-input tasks, the adaptation requests a Python function with a specified input/output interface and verifies recorded test cases. This creates a reproducible transfer task while changing the original submission format; results are reported as adapted-task measurements, not official leaderboard scores. HumanEval+ retains its own official correctness bridge. Protocol-specific extraction is shared across models. A writing or mathematical-reasoning extension would require a defensible correctness verifier and a corresponding definition of solution classes; AST richness is not reused outside code.

F.3 MODEL SIZE AND ARCHITECTURE

Table 12 uses Qwen2.5-Coder at 1.5B, 3B, and 7B, plus DeepSeek-Coder-6.7B-Instruct (Guo et al., 2024) as an additional decoder family. This separates scale within a family from a family change. All new numerical cells remain pending. Model revisions, tokenizer templates, actual target modules, precision, and device resources are recorded by each run. The mathematical framework requires neither a fixed hidden size nor a prescribed number of strategies; the implementation requires compatible native K/V linear maps.

G RESOURCES, SAMPLE LENGTH, AND EXECUTION RECORDS

The completed arms match 291 synthesis prompts, 16 raw candidates per prompt per round, five rounds, and one student epoch per round. This yields 4,656 training records and 291 optimizer

Table 12: Model-scale and family matrix on MBPP. Cells are pass@1 / pass@64 / correct AST richness@64 after five rounds. The first row contains historical results; expanded-protocol runs retain separate identifiers.

Model	Initial	Plain	SPECTRUM
Qwen2.5-Coder-1.5B-Instruct	37.94/72.00/12.80	41.45/70.40/8.51	40.33/72.60/11.51
Qwen2.5-Coder-3B-Instruct	x	x	x
Qwen2.5-Coder-7B-Instruct	x	x	x
DeepSeek-Coder-6.7B-Instruct	x	x	x

steps per round, or 23,280 records and 1,455 steps across the loop. Actual generated lengths differ. Resource comparisons therefore report candidate counts, generated tokens, wall time, and peak memory separately, without declaring equal FLOPs.

The updated suite exports calibration, generation, training, verification, and evaluation resources by round, together with sample-length and truncation counts. GPU elapsed time is distinct from summed wall time; peak allocated and reserved memory are reported with their backend and device. UA-RL’s strategy judge has a separate call and token ledger, including its model and endpoint configuration without credentials. Model training and generated-code verification should run in an appropriately isolated execution environment; verifier outcomes remain auditable from saved records.

H EVIDENCE SCOPE

The completed evidence concerns five-round MBPP learning and native-student evaluation. Independent-seed confirmation, further generator controls, transfer benchmarks, and larger models are specified separately, with unavailable measurements marked x. These extensions distinguish robustness, mechanism, and scope without pooling incompatible evaluation protocols.